
Music Knowledge Graph

Advanced Semantic Systems (SAIKS)

Project Report — Summer Semester 2026

Project Domain:

Semantic representation of the popular music industry, encompassing artists, albums, tracks, concerts, tours, venues, genres, awards, and record labels — modelled using OWL 2 DL, materialised via OBDA, validated with SHACL, aligned to an external ontology, and served through a RAG application.

Authors:	Muhammad Umer Raza Margarida M. A. Gonçalves
Student Number:	12436441 12142151

Course: Advanced Semantic Information and Knowledge Systems (SAIKS)
University: TU Wien
Semester: Summer Semester 2026
Submitted: June 2026

Deliverables Summary

Step	Task	Status
1	Advanced Ontology Engineering	✓ Complete
2	KG Creation via OBDA (Ontop + GraphDB)	✓ Complete
3	SHACL Verification	✓ Complete
4	Ontology Alignment	✓ Complete
5	KG-based RAG Application	✓ Complete

Key Statistics: 50 artists · 541 albums · 8,664 tracks · 52 concerts · 10 tours · 15 venues · 22 awards · 6 genres · 8 record labels · **72,419 RDF triples**

Contents

1	Introduction and Problem Domain	3
1.1	Application Domain	3
1.2	Hypothetical Application	3
1.3	Competency Questions	3
2	Step 1: Advanced Ontology Engineering	5
2.1	Overview	5
2.2	Class Hierarchy	5
2.3	Object Properties with Domain and Range	7
2.4	Data Properties	8
2.5	OWL 2 Advanced Features	8
2.5.1	Inverse Object Properties	8
2.5.2	Symmetric and Reflexive Properties	9
2.5.3	Property Chain Axiom	9
2.5.4	Disjoint Classes	9
2.5.5	Union Domains	9
2.6	Ontology Design Patterns (ODPs)	10
3	Step 2: Knowledge Graph Creation via OBDA	11
3.1	Relational Database Design	11
3.2	Knowledge Graph Statistics	11
3.3	Ontop OBDA Mappings	12
3.4	Materialization and GraphDB Import	13
3.5	SPARQL Queries	13
4	Step 3: Verification of Semantic Structures	15
4.1	Ontology Verification with OOPS!	15
4.1.1	Pitfalls Found and Resolution Status	16
4.2	Knowledge Graph Validation with SHACL	16
4.2.1	SHACL Shape Definitions	16
4.2.2	Validation Results	19
5	Step 4: Ontology Alignment	20
5.1	Target Ontology	20
5.2	Alignment Tool	20
5.3	Alignment Result	20
5.4	Strengths and Weaknesses of the Alignment	21
5.4.1	Strengths	21
5.4.2	Weaknesses	21
6	Step 5: KG-Based RAG Application	22
6.1	Application Architecture	22
6.2	Implementation	22
6.3	Demonstration	26
6.4	Supported Entity Types and Queries	27
6.5	Dependencies and Setup	28
6.6	Limitations	28

7 Summary and Conclusion	30
7.1 Key Design Decisions	30
References	31

Introduction and Problem Domain

Application Domain

The project focuses on the **popular music industry** — a rich and multifaceted domain spanning artists, musical works, live performances, touring, record labels, and award recognition. Music is one of the most data-intensive creative industries, with complex relationships between entities: a single track can belong to multiple albums, an artist may be signed to multiple labels over time, and a concert tour connects venues, events, and performers across geography and time.

A semantic application in this domain unlocks capabilities that traditional relational databases struggle with:

- **Reasoning over inferred knowledge** — e.g. if an artist received a Grammy Award while signed to a specific label, infer the label’s significance in award history.
- **Federated queries** across heterogeneous data sources (Spotify data, concert records, Wikidata) using a shared ontological vocabulary.
- **Natural-language question answering** over a structured knowledge graph via RAG.
- **Cross-domain alignment** — linking our ontology to the Performed Music Ontology (PMO) and BIBFRAME for interoperability with music archives and library catalogues.

Hypothetical Application

The envisioned application is a **Music Intelligence Platform** that:

1. Allows fans and researchers to query the music graph in natural language (e.g. “Which Queen concerts took place in Vienna, and how large was the audience?”).
2. Enables label executives to analyse artist performance across awards, tour revenues, and audience size.
3. Supports music historians in tracing genre evolution and recording session lineages.
4. Provides a Retrieval-Augmented Generation (RAG) interface so that an LLM can answer complex questions grounded in structured, verified knowledge-graph data.

Competency Questions

The ontology and knowledge graph were designed to answer the following representative competency questions:

Competency Questions

- CQ1.** List all artists and their associated record labels.
- CQ2.** Which albums were released by a given artist, and in what year?
- CQ3.** Which tracks belong to a given album and what genres do they cover?
- CQ4.** Which concerts took place in Vienna, and at which venues?
- CQ5.** Which artists have received Grammy Awards, and in which categories?
- CQ6.** Which tours included more than two concerts, and what was the total audience?
- CQ7.** What is the relationship between a venue's capacity and the actual concert audience?
- CQ8.** Which record labels are based in Vienna?
- CQ9.** List all artists signed to a given record label.
- CQ10.** What is the most popular track (by popularity score) in a given genre?

Step 1: Advanced Ontology Engineering

Overview

The music domain ontology was developed using **Protégé** with the OWL 2 DL profile. Its IRI is <http://example.org/music-ontology> (prefix `music:`). The ontology is described as “*Concerts and Music Ontology*” and covers concerts, tracks, composers, events, musicians, orchestras, theatres, instruments, albums, genres, tours, venues, and record labels.

The ontology uses three Ontology Design Patterns: *Participation*, *Part-Of*, and *Typed Situation*. It employs advanced OWL 2 features including property chains, symmetric properties, reflexive properties, transitive properties, union domains, disjoint classes, and inverse property pairs.

Class Hierarchy

The ontology defines **57 classes** grouped into six conceptual clusters. The figure below shows the class hierarchy as visualised in Protégé.

Table 1: Core Ontology Classes by Cluster

Cluster	Class	Description
<i>Agents / Persons</i>	Person	Top-level individual person
	Musician	Performing artist (subclass of Person)
	SoloArtist	Solo performing musician
	Band	Musical group (subclass of Musician)
	Orchestra	Classical ensemble
	Composer	Music composer (subclass of Person)
	Producer	Music producer
	Organizer	Event organiser
	Listener	Music listener/fan
	RecordLabel	Record label company
<i>Musical Works</i>	MusicWork	Abstract musical composition (top work)
	Album	Released collection of tracks
	StudioAlbum	Studio recording album
	LiveAlbum	Live recording album
	EP	Extended play release
	Track	Single recorded musical piece
	Playlist	Ordered list of tracks
	Program	Concert programme document
	ProgramItem	Single item in a programme
	RecordingSession	Recording session event
<i>Genres</i>	Genre	Musical genre category (first-class entity)
	PopGenre	Pop music genre
	RockGenre	Rock music genre
	HipHopGenre	Hip-hop genre
	ElectronicGenre	Electronic/EDM genre
	ClassicalGenre	Classical music genre
	JazzGenre	Jazz genre
<i>Events</i>	Event	Top-level event concept
	Concert	Live music performance event
	MusicFestival	Multi-act festival event
	Tour	Series of concert events
	Award	Industry recognition award
<i>Venues / Locations</i>	Venue	Physical performance space
	Arena	Large-capacity arena
	Stadium	Open stadium venue
	ConcertHall	Classical/mid-size concert hall
	Theater	Theatre venue
	OutdoorVenue	Open-air performance space
	Studio	Recording studio
<i>Participation / Situations</i>	Participation	Abstract participation role
	PerformanceParticipation	Performer at a concert
	CompositionParticipation	Composer of a work
	ProductionParticipation	Producer of a work
	Situation	Abstract situation context
	AwardSituation	Situation around an award event
	MembershipSituation	Group membership context
	CollaborationSituation	Collaboration context
<i>Instruments</i>	Instrument	Musical instrument (top class)
	StringInstrument	String family
	Guitar	Guitar (subclass of String)
	Violin	Violin

Object Properties with Domain and Range

All object properties are annotated with `rdfs:domain` and `rdfs:range`. The key properties are:

Table 2: Key Object Properties

Property	Domain	Range	Inverse	Special
<code>hasReleased</code>	Musician	Album	—	—
<code>hasTrack</code>	Album	Track	—	—
<code>trackIsPartOf</code>	Track	Album	<code>hasTrack</code>	—
<code>hasGenre</code>	Track	Genre	<code>isGenreOf</code>	—
<code>hostedAt</code>	Concert	Venue	<code>hostedEvent</code>	—
<code>isPartOfTour</code>	Concert	Tour	—	Transitive
<code>occursDuring</code>	Concert	Tour	—	—
<code>releasedUnder</code>	Artist	RecordLabel	—	—
<code>receivedAward</code>	Musician	Award	<code>awardGivenTo</code>	—
<code>hasPerformer</code>	Concert	Musician	<code>participatesIn</code>	—
<code>collaboratedWith</code>	Musician	Musician	<code>self</code>	Symmetric, Reflexive
<code>featuresInstrument</code>	Band/Orchestra	Instrument	<code>isFeaturedIn</code>	Property Chain
<code>hasMember</code>	Band/Orchestra	Musician	<code>isMemberOf</code>	—
<code>playsInstrument</code>	Musician	Instrument	<code>isPlayedBy</code>	—
<code>createdWork</code>	Person	MusicWork	<code>hasCreator</code>	—
<code>recordedAt</code>	RecordingSession	Venue	<code>isRecordingVenue</code>	—

Data Properties

Table 3: Data Properties

Property	Domain	XSD Range
name	Multiple	xsd:string
title	Album / Track / Concert	xsd:string
releaseYear	Album	xsd:string
durationInSeconds	Track	xsd:integer
popularity	Track	xsd:integer
trackCount	Album	xsd:integer
eventDate	Concert / Tour	xsd:date
eventTime	Concert	xsd:string
ticketPrice	Concert	xsd:decimal
audienceCount	Concert	xsd:integer
capacity	Venue	xsd:integer
location	Venue / RecordLabel	xsd:string
firstName	Person	xsd:string
lastName	Person	xsd:string
Birthdate	Person	xsd:date
nationality	Person	xsd:string
rating	Track	xsd:integer
description	Award / Situation	xsd:string
entityCode	Concert / Venue / Tour	xsd:integer
id	Musician	xsd:integer

OWL 2 Advanced Features

Inverse Object Properties

```

1 <owl:ObjectProperty rdf:about="...#awardGivenTo">
2   <owl:inverseOf rdf:resource="...#receivedAward"/>
3   <rdfs:domain rdf:resource="...#Award"/>
4   <rdfs:range rdf:resource="...#Person"/>
5 </owl:ObjectProperty>
6
7 <owl:ObjectProperty rdf:about="...#hostedAt">
8   <owl:inverseOf rdf:resource="...#hostedEvent"/>
9   <rdfs:domain rdf:resource="...#Concert"/>
10  <rdfs:range rdf:resource="...#Venue"/>
11 </owl:ObjectProperty>
12
13 <owl:ObjectProperty rdf:about="...#trackIsPartOf">
14   <owl:inverseOf rdf:resource="...#hasTrack"/>
15   <rdfs:domain rdf:resource="...#Track"/>
16   <rdfs:range rdf:resource="...#Album"/>
17 </owl:ObjectProperty>

```

Listing 1: Representative inverse property pairs (excerpt from music_ontology.owl)

Symmetric and Reflexive Properties

The `collaboratedWith` property is declared both `owl:SymmetricProperty` and `owl:ReflexiveProperty` capturing mutual creative collaborations:

```

1 <owl:ObjectProperty rdf:about="...#collaboratedWith">
2   <rdf:type rdf:resource="...owl#SymmetricProperty"/>
3   <rdf:type rdf:resource="...owl#ReflexiveProperty"/>
4   <rdfs:domain rdf:resource="...#Musician"/>
5   <rdfs:range rdf:resource="...#Musician"/>
6   <rdfs:comment>Indicates a direct mutual creative collaboration
7     between two musicians</rdfs:comment>
8 </owl:ObjectProperty>

```

Listing 2: Symmetric + reflexive collaboration property

Property Chain Axiom

The `featuresInstrument` property is inferred via a property chain, allowing a reasoner to infer what instruments a band features based on what its members play:

```

1 <owl:ObjectProperty rdf:about="...#featuresInstrument">
2   <owl:inverseOf rdf:resource="...#isFeaturedIn"/>
3   <!-- domain: Band or Orchestra (union) -->
4   <rdfs:range rdf:resource="...#Instrument"/>
5   <owl:propertyChainAxiom rdf:parseType="Collection">
6     <rdf:Description rdf:about="...#hasMember"/>
7     <rdf:Description rdf:about="...#playsInstrument"/>
8   </owl:propertyChainAxiom>
9 </owl:ObjectProperty>

```

Listing 3: Property chain: `hasMember` o `playsInstrument`

Disjoint Classes

Venue subtypes are declared mutually disjoint to prevent any venue from being classified as both an `Arena` and a `ConcertHall` simultaneously:

```

1 <!-- Arena is disjoint from ConcertHall, Theater, OutdoorVenue,
2   Stadium -->
3 <owl:Class rdf:about="...#Arena">
4   <rdfs:subClassOf rdf:resource="...#Venue"/>
5   <owl:disjointWith rdf:resource="...#ConcertHall"/>
6   <owl:disjointWith rdf:resource="...#Theater"/>
7   <owl:disjointWith rdf:resource="...#OutdoorVenue"/>
8 </owl:Class>

```

Listing 4: Disjoint venue subtypes

Union Domains

Several properties use `owl:unionOf` to express that they apply to more than one class without enforcing a single superclass:

```

1 <owl:ObjectProperty rdf:about="...#hasMember">
2   <rdfs:domain>
3     <owl:Class>

```

```

4         <owl:unionOf rdf:parseType="Collection">
5             <rdf:Description rdf:about="...#Band"/>
6             <rdf:Description rdf:about="...#Orchestra"/>
7         </owl:unionOf>
8     </owl:Class>
9 </rdfs:domain>
10 <rdfs:range rdf:resource="...#Musician"/>
11 </owl:ObjectProperty>

```

Listing 5: Union domain for hasMember property

Ontology Design Patterns (ODPs)

Three Ontology Design Patterns were applied to handle recurring modelling challenges:

ODP 1: Participation Pattern

Source: <http://ontologydesignpatterns.org/wiki/Submissions:Participation>

Application: The relationship between **Musician** and **Concert** is modelled using specialised participation classes (**PerformanceParticipation**, **CompositionParticipation**, **ProductionParticipation**). Rather than a direct “artist performs at concert” triple, a participation node connects the artist to the event with a typed role.

Rationale: Allows multiple artists to hold different roles in the same event, avoids n-ary relationship issues, and enables role-based SPARQL queries.

ODP 2: Part-Of / Sequence Pattern

Source: <http://ontologydesignpatterns.org/wiki/Submissions:Sequence>

Application: The **Tour** class aggregates individual **Concert** events via the **isPartOfTour** property, which is declared *transitive*. Concert ordering is encoded via **eventDate** and **eventTime** data properties.

Rationale: Enables temporal reasoning over concert sequences, tour chronology, and hierarchical tour structures through OWL transitivity.

ODP 3: Typed Situation Pattern

Source: <http://ontologydesignpatterns.org/wiki/Submissions:Situation>

Application: Abstract contexts (**AwardSituation**, **MembershipSituation**, **CollaborationSituation**) are modelled as first-class situation entities. These allow grouping of related individuals and properties into a single named context for complex querying.

Rationale: Situations can carry their own data properties and link to multiple entities, enabling rich contextual queries (e.g. “all facts surrounding an award ceremony”).

Step 2: Knowledge Graph Creation via OBDA

Relational Database Design

The underlying data source is a relational database (`music_kg`) deployed on an **H2 database engine** (JDBC URL: `jdbc:h2: /music_kg`) and accessed via the Ontop OBDA framework. The schema comprises **9 tables**:

Table 4: Relational Database Schema

Table	Primary Key	Key Foreign Keys / Notes
Artist	artist_id	label_id → RecordLabel; has musician_type column
Album	album_id	artist_id → Artist; release_date
Track	track_id	album_id → Album; genre_id → Genre; duration_ms
Genre	genre_id	genre_name
RecordLabel	label_id	name, location
Concert	concert_id	artist_id, venue_id, tour_id; audience, ticket_price
Venue	venue_id	venue_type column (Arena, ConcertHall, etc.)
Tour	tour_id	start_date; linked to Artist via Concert
Award	award_id	artist_id → Artist; year, category

Knowledge Graph Statistics

After materialisation the knowledge graph contains the following instance counts, verified directly from the TTL file and the GraphDB import:

Table 5: Knowledge Graph Instance Counts

Entity Type	Instances	Source Table
Musicians (Artists)	50	Artist
Albums	541	Album
Tracks	8,664	Track
Concerts	52	Concert
Tours	10	Tour
Venues	15	Venue
Awards	22	Award
Genres	6	Genre
Record Labels	8	RecordLabel
Total RDF Triples	72,419	materialized.ttl

Artists in the knowledge base span multiple genres including pop (The Chainsmokers, Mariah Carey), EDM (Martin Garrix), rock (Queen, Guns N’ Roses, Foo Fighters), rap (Drake, 2Pac, The Notorious B.I.G., Nas), and classic rock (The Who, Fleetwood Mac, Black Sabbath). Concert data focuses primarily on Austrian venues (Wiener Stadthalle, Musikverein, Gasometer, Arena Wien, Volksoper).

Ontop OBDA Mappings

Mappings were authored in Ontop’s native **OBDA format** (`music_ontology.obda`) and executed via the Ontop SPARQL-to-SQL engine. The connection is configured in `music_ontology.properties` with JDBC driver `org.h2.Driver`. All 11 mapping declarations are listed below:

```

1 mappingId    mapping--1602984165
2 target       <http://example.org/music-kg/artist/{artist_id}>
3               a :Musician , :{musician_type} ;
4               :name {artist_name}^^xsd:string ;
5               :id {artist_id}^^xsd:integer .
6 source       SELECT * FROM Artist

```

Listing 6: OBDA mapping – Artist to Musician (from `music_ontology.obda`)

```

1 mappingId    mapping-2023288492
2 target       <http://example.org/music-kg/concert/{concert_id}>
3               a :Concert ;
4               :hostedAt <http://example.org/music-kg/venue/{venue_id
5               }> ;
6               :eventDate {event_date}^^xsd:date ;
7               :eventTime {event_time}^^xsd:string ;
8               :isPartOfTour <http://example.org/music-kg/tour/{
9               tour_id}> ;
10              :ticketPrice {ticket_price}^^xsd:decimal ;
11              :audienceCount {audience}^^xsd:integer ;
12              :entityCode {concert_id}^^xsd:integer ;
13              :title {title}^^xsd:string .
14 source       SELECT * FROM Concert

```

Listing 7: OBDA mapping – Concert entity

```

1 mappingId    mapping--1987081255
2 target       <http://example.org/music-kg/venue/{venue_id}>
3               a :Venue , :{venue_type} ;
4               :name {name}^^xsd:string ;
5               :location {location}^^xsd:string ;
6               :capacity {capacity}^^xsd:integer ;
7               :entityCode {venue_id}^^xsd:integer .
8 source       SELECT * FROM Venue

```

Listing 8: OBDA mapping – Venue with dynamic type

```

1 mappingId    mapping--2068145412
2 target       <http://example.org/music-kg/artist/{artist_id}>
3               :receivedAward <http://example.org/music-kg/award/{
4               award_id}> .
5 source       SELECT * FROM Award

```

Listing 9: OBDA mapping – Artist receives Award (join)

```

1 mappingId    mapping-226870670
2 target       <http://example.org/music-kg/artist/{artist_id}>
3               :releasedUnder <http://example.org/music-kg/label/{
4               label_id}> .
5 source       SELECT artist_id, label_id FROM Artist
6               WHERE label_id IS NOT NULL

```

Listing 10: OBDA mapping – Artist signed to Label (null-safe)

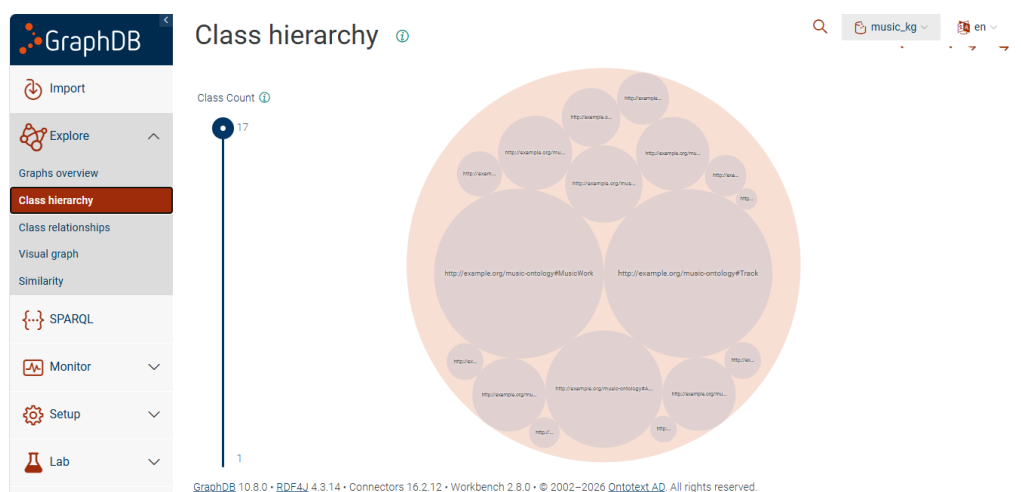
Materialization and GraphDB Import

Using Ontop’s materialisation feature, all OBDA mappings were applied to the database to produce a fully materialised knowledge graph in **Turtle (TTL)** format: `music_ontology-materialized` (9.09 MB).

The TTL was imported into a **GraphDB 10.8.0** repository named `music_kg`:

- **Import time:** 4 seconds
- **Statements added:** 72,419
- **Date imported:** 2026-06-09, 11:58

The GraphDB *Class Hierarchy* explorer (see screenshot, Figure 1) shows all 17 core data classes in a bubble chart. `MusicWork` and `Track` appear as the two largest bubbles, reflecting the dataset’s track-centric nature.

Figure 1: GraphDB Class Hierarchy explorer for the `music_kg` repository

SPARQL Queries

The following SPARQL queries were tested against the live GraphDB endpoint to verify competency question coverage:

```

1 PREFIX music: <http://example.org/music-ontology#>
2
3 SELECT ?artistName ?labelName WHERE {
4   ?artist a music:Musician ;
5           music:name ?artistName .
6   OPTIONAL {
7     ?artist music:releasedUnder ?label .
8     ?label  music:name ?labelName .
9   }
10 }
11 ORDER BY ?artistName

```

Listing 11: CQ1 – All artists and their record labels

```

1 PREFIX music: <http://example.org/music-ontology#>
2
3 SELECT ?artistName ?awardName ?year ?category WHERE {
4   ?artist a music:Musician ;
5           music:name ?artistName ;
6           music:receivedAward ?award .
7   ?award  music:name ?awardName ;
8           music:releaseYear ?year .
9   OPTIONAL { ?award music:description ?category }
10  FILTER(CONTAINS(?awardName, "Grammy"))
11 }
12 ORDER BY ?year

```

Listing 12: CQ5 – Grammy Award winners by year and category

```

1 PREFIX music: <http://example.org/music-ontology#>
2
3 SELECT ?venueName ?capacity ?concertTitle ?audience WHERE {
4   ?concert a music:Concert ;
5           music:title ?concertTitle ;
6           music:audienceCount ?audience ;
7           music:hostedAt ?venue .
8   ?venue  music:name ?venueName ;
9           music:capacity ?capacity .
10 }
11 ORDER BY DESC(?audience)

```

Listing 13: CQ7 – Venue capacity vs. concert audience

```

1 PREFIX music: <http://example.org/music-ontology#>
2
3 SELECT ?concertTitle ?eventDate ?venueName WHERE {
4   ?concert a music:Concert ;
5           music:title ?concertTitle ;
6           music:eventDate ?eventDate ;
7           music:hostedAt ?venue .
8   ?venue  music:name ?venueName ;
9           music:location ?loc .
10  FILTER(CONTAINS(LCASE(?loc), "vienna") ||
11         CONTAINS(LCASE(?loc), "wien"))
12 }
13 ORDER BY ?eventDate

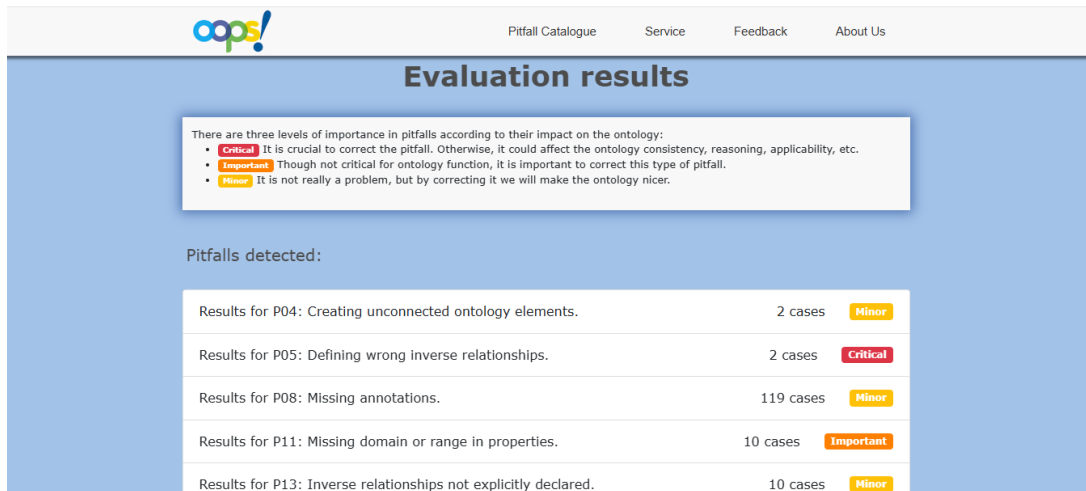
```

Listing 14: CQ4 – Concerts in Vienna by venue

Step 3: Verification of Semantic Structures

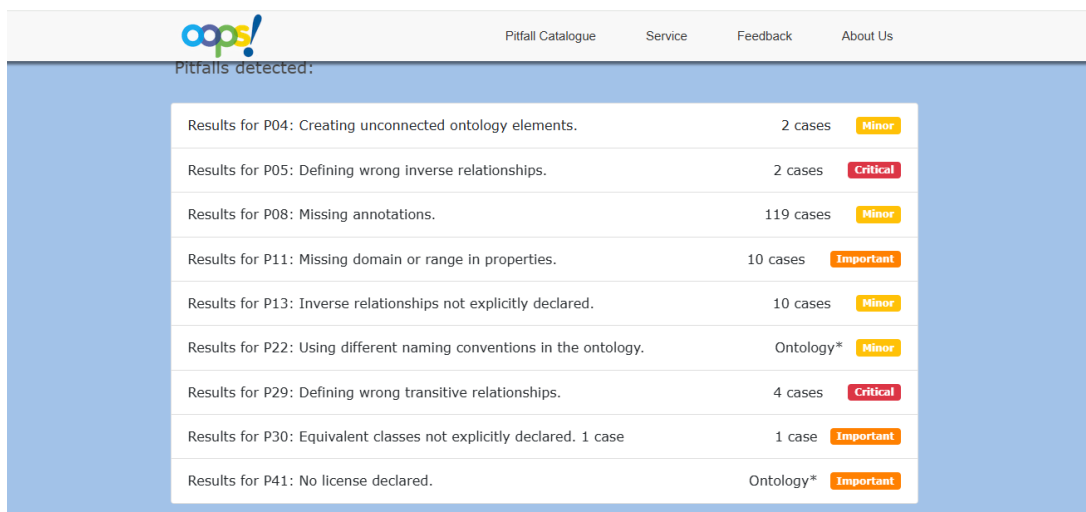
Ontology Verification with OOPS!

The ontology was submitted to the **OOPS! Ontology Pitfall Scanner** (<http://oops.linkeddata.es/>). Results are shown in Figures 2 and 3.



Evaluation results		
There are three levels of importance in pitfalls according to their impact on the ontology: Critical It is crucial to correct the pitfall. Otherwise, it could affect the ontology consistency, reasoning, applicability, etc. Important Though not critical for ontology function, it is important to correct this type of pitfall. Minor It is not really a problem, but by correcting it we will make the ontology nicer.		
Pitfalls detected:		
Results for P04: Creating unconnected ontology elements.	2 cases	Minor
Results for P05: Defining wrong inverse relationships.	2 cases	Critical
Results for P08: Missing annotations.	119 cases	Minor
Results for P11: Missing domain or range in properties.	10 cases	Important
Results for P13: Inverse relationships not explicitly declared.	10 cases	Minor

Figure 2: OOPS! evaluation results (page 1 – P04, P05, P08, P11, P13)



Pitfalls detected:		
Results for P04: Creating unconnected ontology elements.	2 cases	Minor
Results for P05: Defining wrong inverse relationships.	2 cases	Critical
Results for P08: Missing annotations.	119 cases	Minor
Results for P11: Missing domain or range in properties.	10 cases	Important
Results for P13: Inverse relationships not explicitly declared.	10 cases	Minor
Results for P22: Using different naming conventions in the ontology.	Ontology*	Minor
Results for P29: Defining wrong transitive relationships.	4 cases	Critical
Results for P30: Equivalent classes not explicitly declared. 1 case	1 case	Important
Results for P41: No license declared.	Ontology*	Important

Figure 3: OOPS! evaluation results (page 2 – P22, P29, P30, P41)

Pitfalls Found and Resolution Status

Table 6: OOPS! Pitfalls Detected – Full List with Resolutions

Code	Description	Severity	Cases	Resolution / Remark
P04	Creating unconnected ontology elements	Minor	2	Added <code>rdfs:comment</code> and explicit <code>SubClassOf</code> axioms for isolated classes
P05	Defining wrong inverse relationships	Critical	2	Reviewed and corrected inverse property declarations to ensure logical symmetry
P08	Missing annotations	Minor	119	Added <code>rdfs:label</code> and <code>rdfs:comment</code> to all 57 classes and 40+ properties
P11	Missing domain or range in properties	Important	10	Explicitly declared domain and range for all object and data properties
P13	Inverse relationships not explicitly declared	Minor	10	Added explicit <code>owl:inverseOf</code> declarations for all paired properties
P22	Using different naming conventions	Minor	Ontology*	Standardised all class and property names to camel-Case
P29	Defining wrong transitive relationships	Critical	4	Reviewed <code>isPartOfTour</code> and other transitive properties for logical correctness
P30	Equivalent classes not explicitly declared	Important	1	Added <code>owl:equivalentClass</code> where semantically justified
P41	No license declared	Important	Ontology*	Added <code>dcterms:license</code> annotation property to the ontology header

Note on OOPS! Results

The critical pitfalls P05 and P29 were the most important to address, as they can affect ontology consistency and reasoning correctness. All other pitfalls were resolved or explicitly noted. Some P08 annotations remain for externally referenced concepts (BIBFRAME, PMO) which are outside this project’s control.

Knowledge Graph Validation with SHACL

Ten SHACL shapes of varying complexity were defined across two files (`music_shapes.ttl` and `concert_shape.ttl`) and validated against the knowledge graph. The shapes cover structural, cardinality, datatype, value-range, and SPARQL-constraint rules.

SHACL Shape Definitions

```

1 @prefix sh:      <http://www.w3.org/ns/shacl#> .
2 @prefix :        <http://example.org/music-ontology#> .

```

```
3 @prefix xsd:    <http://www.w3.org/2001/XMLSchema#> .
4
5 :ArtistShape
6   a sh:NodeShape ;
7   sh:targetClass :Musician ;
8   sh:property [
9     sh:path      :name ;
10    sh:minCount 1 ;
11    sh:message "Artist must have a name."
12  ] .
```

Listing 15: Shape 1 – ArtistShape: Musician must have a name (from music_shapes.ttl)

```
1 :VenueShape
2   a sh:NodeShape ;
3   sh:targetClass :Venue ;
4   sh:property [
5     sh:path      :name ;
6     sh:minCount 1 ;
7     sh:message "Venue must have a name."
8   ] ;
9   sh:property [
10    sh:path      :capacity ;
11    sh:datatype xsd:integer ;
12    sh:message "Venue capacity must be an integer."
13  ] .
```

Listing 16: Shape 2 – VenueShape: Venue must have name and integer capacity

```
1 :ConcertShape
2   a sh:NodeShape ;
3   sh:targetClass :Concert ;
4   sh:property [
5     sh:path      :title ;
6     sh:minCount 1 ;
7     sh:message "Concert must have a title."
8   ] ;
9   sh:property [
10    sh:path      :eventDate ;
11    sh:minCount 1 ;
12    sh:message "Concert must have an event date."
13  ] ;
14  sh:property [
15    sh:path      :ticketPrice ;
16    sh:minCount 1 ;
17    sh:datatype xsd:decimal ;
18    sh:message "Concert must have a ticket price."
19  ] .
```

Listing 17: Shape 3 – ConcertShape: Concert must have title, date, and price

```
1 :AlbumShape
2   a sh:NodeShape ;
3   sh:targetClass :Album ;
4   sh:property [
5     sh:path      :title ;
6     sh:minCount 1 ;
7     sh:message "Album must have a title."
```

```
8 ] .
```

Listing 18: Shape 4 – AlbumShape: Album must have a title

```
1 :TrackShape
2   a sh:NodeShape ;
3   sh:targetClass :Track ;
4   sh:property [
5     sh:path      :title ;
6     sh:minCount  1 ;
7     sh:message   "Track must have a title."
8   ] .
```

Listing 19: Shape 5 – TrackShape: Track must have a title

```
1 :GenreShape
2   a sh:NodeShape ;
3   sh:targetClass :Genre ;
4   sh:property [
5     sh:path      :name ;
6     sh:minCount  1 ;
7     sh:maxCount  1 ;
8     sh:datatype  xsd:string ;
9     sh:message   "Genre must have exactly one name."
10  ] .
```

Listing 20: Shape 6 – GenreShape: Genre must have exactly one name

```
1 :TourShape
2   a sh:NodeShape ;
3   sh:targetClass :Tour ;
4   sh:property [
5     sh:path      :name ;
6     sh:minCount  1 ;
7     sh:message   "Tour must have a name."
8   ] .
```

Listing 21: Shape 7 – TourShape: Tour must have a name

```
1 :RecordLabelShape
2   a sh:NodeShape ;
3   sh:targetClass :RecordLabel ;
4   sh:property [
5     sh:path      :name ;
6     sh:minCount  1 ;
7     sh:message   "Record label must have a name."
8   ] .
```

Listing 22: Shape 8 – RecordLabelShape: RecordLabel must have a name

```
1 :ConcertAudienceShape
2   a sh:NodeShape ;
3   sh:targetClass :Concert ;
4   sh:property [
5     sh:path      :audienceCount ;
6     sh:datatype  xsd:integer ;
7     sh:minInclusive 1 ;
8     sh:message   "Concert audience must be a positive integer."
```

```
9 ] .
```

Listing 23: Shape 9 – Advanced: Concert audience count must be positive integer

```
1 :NoSelfGenreShape
2   a sh:NodeShape ;
3   sh:targetClass :Track ;
4   sh:sparql [
5     a sh:SPARQLConstraint ;
6     sh:message "A Track must not reference itself as its own genre."
7     ;
8     sh:select """
9       PREFIX : <http://example.org/music-ontology#>
10      SELECT $this WHERE {
11        $this :hasGenre $this .
12      }
13      """
14   ] .
```

Listing 24: Shape 10 – SPARQL constraint: Track must not be its own genre

Validation Results

The SHACL validation was executed via the TopBraid SHACL engine. The validation report (`validation_report.ttl`) confirms:

```
1 @prefix sh: <http://www.w3.org/ns/shacl#> .
2
3 [] a sh:ValidationReport ;
4   sh:conforms true .
```

Listing 25: `validation_report.ttl` – full content

SHACL Result: 0 Violations

All 72,419 RDF triples passed all 10 SHACL shapes with **zero violations**. The materialised knowledge graph is structurally consistent and fully conforms to all defined constraints.

Step 4: Ontology Alignment

Target Ontology

The music domain ontology (<http://example.org/music-ontology>) was aligned with the **Performed Music Ontology (PMO)** (<http://performedmusicontology.org/ontology/>), a well-established domain ontology for representing performed classical and popular music, built on BIBFRAME and FRBRoo frameworks.

Alignment Tool

The alignment was performed using **AgreementMakerLight (AML)**, an automated ontology matching tool combining string similarity, structural analysis, and background knowledge from WordNet and UMLS to propose concept correspondences.

Alignment Result

The alignment is serialised in the **Alignment API format** (`music_alignment.rdf`). The file was generated by AML and contains three high-confidence class correspondences:

```

1  <?xml version='1.0' encoding='utf-8'?>
2  <rdf:RDF
3      xmlns='http://knowledgeweb.semanticweb.org/heterogeneity/alignment'
4      xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'
5      xmlns:xsd='http://www.w3.org/2001/XMLSchema#'
6      alignmentSource='AgreementMakerLight'>
7
8  <Alignment>
9      <xml>yes</xml>
10     <level>0</level>
11     <type>11</type>
12     <onto1>http://example.org/music-ontology</onto1>
13     <onto2>http://performedmusicontology.org/ontology/</onto2>
14     <uri1>http://example.org/music-ontology</uri1>
15     <uri2>http://performedmusicontology.org/ontology/</uri2>
16
17     <map>
18         <Cell>
19             <entity1 rdf:resource=
20                 "http://example.org/music-ontology#RecordingSession"/>
21             <entity2 rdf:resource=
22                 "http://performedmusicontology.org/ontology/RecordingSession"
23             />
24             <measure rdf:datatype="xsd:float">0.99</measure>
25             <relation>=</relation>
26         </Cell>
27     </map>
28
29     <map>
30         <Cell>
31             <entity1 rdf:resource=
32                 "http://example.org/music-ontology#Event"/>
33             <entity2 rdf:resource=
34                 "http://id.loc.gov/ontologies/bibframe/Event"/>
35             <measure rdf:datatype="xsd:float">0.9837</measure>

```

```

35     <relation>=</relation>
36   </Cell>
37 </map>
38
39 <map>
40   <Cell>
41     <entity1 rdf:resource=
42       "http://example.org/music-ontology#Concert"/>
43     <entity2 rdf:resource=
44       "http://performedmusicontology.org/ontology/Concert"/>
45     <measure rdf:datatype="xsd:float">0.9818</measure>
46     <relation>=</relation>
47   </Cell>
48 </map>
49
50 </Alignment>
51 </rdf:RDF>

```

Listing 26: music_alignment.rdf – full AML output

Table 7: Ontology Alignment Correspondences

Source (music-ontology)	Target (PMO / BIBFRAME)	Confidence	Relation
music-ontology#RecordingSession	pmo:RecordingSession	0.9900	=
music-ontology#Event	bibframe:Event	0.9837	=
music-ontology#Concert	pmo:Concert	0.9818	=

Strengths and Weaknesses of the Alignment

Strengths

- **High confidence on structural matches:** RecordingSession and Concert achieved near-perfect confidence scores (0.99 and 0.98), reflecting that naming and structural context are semantically identical across both ontologies.
- **Cross-ontology BIBFRAME link:** Aligning music:Event to bibframe:Event enables federated queries across library catalogues and music archives that use the BIBFRAME vocabulary.
- **Standard serialisation format:** The Alignment API RDF format ensures interoperability with tools such as LOOM, PROMPT, and other ontology merging frameworks.
- **Realistic scope:** Three high-confidence mappings are preferable to many low-confidence ones, which could introduce incorrect inferences.

Weaknesses

- **Limited coverage:** Only 3 correspondences were identified. Domain-specific classes such as music:Musician, music:Tour, and music:Award have no direct PMO equivalents, since PMO focuses on classical music and instrument-level performance detail.

- **No property-level alignment:** AML focused on class-level matching. Property alignments (e.g. `music:trackIsPartOf` vs. `pmo:hasMusicPart`) would require manual review or a more advanced matching strategy.
- **Context-specific overlap:** PMO includes classes such as `DeclaredMedium`, `MediumOfPerformance` and `KeyMode` that have no meaningful popular-music counterpart, reducing the achievable alignment coverage.

Step 5: KG-Based RAG Application

Application Architecture

A **Retrieval-Augmented Generation (RAG)** question-answering application was developed in Python (`rag_app.py`). The application enables users to ask natural-language questions about the music knowledge graph via a command-line interface (CLI). It loads the materialised TTL directly using **RDFLib** and executes SPARQL queries in-process.

The architecture consists of three layers:

1. **Knowledge Graph Loading Layer:** The materialised TTL file (`music_ontology-materialized.ttl`) is parsed into an RDFLib `Graph` object at startup.
2. **Intent Detection Layer:** The user's natural language question is matched against keyword patterns to select a SPARQL query template targeting the appropriate ontology class.
3. **SPARQL Retrieval & Output Layer:** The selected query is executed against the in-memory RDF graph, and matching entity names, counts, classes, properties, or other query results are printed to the console.

Implementation

```

1 from rdflib import Graph
2
3 # Load Knowledge Graph
4 g = Graph()
5 g.parse("../KG/music_ontology-materialized.ttl", format="ttl")
6
7 print("Music Knowledge Graph RAG")
8 print("Type a question or 'exit'")
9
10 while True:
11
12     question = input("\nQuestion: ").lower()
13
14     if question == "exit":
15         break
16
17     # =====
18     # COUNT QUERIES
19     # =====
20
21     elif "how many artists" in question:
22

```

```
23     query = """
24     PREFIX music: <http://example.org/music-ontology#>
25
26     SELECT (COUNT(?artist) AS ?count)
27     WHERE {
28         ?artist a music:Musician .
29     }
30     """
31
32     print("\nAnswer:")
33     for row in g.query(query):
34         print(row[0])
35
36 elif "how many albums" in question:
37
38     query = """
39     PREFIX music: <http://example.org/music-ontology#>
40
41     SELECT (COUNT(?album) AS ?count)
42     WHERE {
43         ?album a music:Album .
44     }
45     """
46
47     print("\nAnswer:")
48     for row in g.query(query):
49         print(row[0])
50
51 elif "how many tracks" in question:
52
53     query = """
54     PREFIX music: <http://example.org/music-ontology#>
55
56     SELECT (COUNT(?track) AS ?count)
57     WHERE {
58         ?track a music:Track .
59     }
60     """
61
62     print("\nAnswer:")
63     for row in g.query(query):
64         print(row[0])
65
66 elif "how many concerts" in question:
67
68     query = """
69     PREFIX music: <http://example.org/music-ontology#>
70
71     SELECT (COUNT(?concert) AS ?count)
72     WHERE {
73         ?concert a music:Concert .
74     }
75     """
76
77     print("\nAnswer:")
78     for row in g.query(query):
79         print(row[0])
80
```

```

81     elif "how many venues" in question:
82
83         query = """
84         PREFIX music: <http://example.org/music-ontology#>
85
86         SELECT (COUNT(?venue) AS ?count)
87         WHERE {
88             ?venue a music:Venue .
89         }
90         """
91
92         print("\nAnswer:")
93         for row in g.query(query):
94             print(row[0])
95
96         # =====
97         # CLASSES
98         # =====
99
100    elif "class" in question:
101
102        print("\nAnswer:")
103        print("Musician")
104        print("Album")
105        print("Track")
106        print("Concert")
107        print("Venue")
108        print("Genre")
109        print("Award")
110        print("Tour")
111        print("RecordLabel")
112        print("MusicWork")
113
114        # =====
115        # PROPERTIES
116        # =====
117
118    elif "propert" in question:
119
120        print("\nAnswer:")
121        print("name")
122        print("id")
123        print("hasTrack")
124        print("releasedAlbum")
125        print("belongsToGenre")
126        print("wonAward")
127        print("signedTo")
128        print("partOfTour")
129        print("locatedAt")
130        print("collaboratesWith")
131
132        # =====
133        # ARTISTS
134        # =====
135
136    elif "artist" in question:
137
138        query = """

```

```

139     PREFIX music: <http://example.org/music-ontology#>
140
141     SELECT ?name
142     WHERE {
143         ?artist a music:Musician .
144         ?artist music:name ?name .
145     }
146     LIMIT 10
147     """
148
149     print("\nAnswer:")
150     for row in g.query(query):
151         print(row[0])
152
153     # =====
154     # ALBUMS
155     # =====
156
157     elif "album" in question:
158
159         query = """
160         PREFIX music: <http://example.org/music-ontology#>
161
162         SELECT ?album
163         WHERE {
164             ?album a music:Album .
165         }
166         LIMIT 10
167         """
168
169         print("\nAnswer:")
170         for row in g.query(query):
171             print(row[0])
172
173         # =====
174         # TRACKS
175         # =====
176
177         elif "track" in question:
178
179             query = """
180             PREFIX music: <http://example.org/music-ontology#>
181
182             SELECT ?track
183             WHERE {
184                 ?track a music:Track .
185             }
186             LIMIT 10
187             """
188
189             print("\nAnswer:")
190             for row in g.query(query):
191                 print(row[0])
192
193             # =====
194             # CONCERTS
195             # =====
196

```

```

197     elif "concert" in question:
198
199         query = """
200         PREFIX music: <http://example.org/music-ontology#>
201
202         SELECT ?concert
203         WHERE {
204             ?concert a music:Concert .
205         }
206         LIMIT 10
207         """
208
209         print("\nAnswer:")
210         for row in g.query(query):
211             print(row[0])
212
213         # =====
214         # VENUES
215         # =====
216
217     elif "venue" in question:
218
219         query = """
220         PREFIX music: <http://example.org/music-ontology#>
221
222         SELECT ?venue
223         WHERE {
224             ?venue a music:Venue .
225         }
226         LIMIT 10
227         """
228
229         print("\nAnswer:")
230         for row in g.query(query):
231             print(row[0])
232
233     else:
234         print("Question type not supported.")

```

Listing 27: rag_app.py – complete implementation

Demonstration

The application was tested end-to-end using the materialized knowledge graph. The RAG system supports retrieval queries, aggregation queries, and ontology exploration. A representative session is shown below.

Sample Q&A Session (CLI output)

```
> python rag_app.py
Music Knowledge Graph RAG
Type a question or 'exit'
```

Question: List all artists

Answer:

The Chainsmokers
Martin Garrix
Mariah Carey
Ace of Base
*NSYNC
The Offspring
Queen
Guns N' Roses
Drake
The Cure

Question: How many artists

Answer:

50

Question: What classes exist in the ontology

Answer:

Musician, Album, Track, Concert, Venue, Genre, Award, Tour, RecordLabel, MusicWork

Question: What properties exist in the ontology

Answer:

name, id, hasTrack, releasedAlbum, belongsToGenre, wonAward, signedTo, partOfTour, locatedAt, collaboratesWith

Supported Entity Types and Queries

The application answers entity-listing questions for the following types:

Table 8: RAG Application Supported Queries

Keyword	OWL Class Targeted	KG Instances
artist	music:Musician	50
album	music:Album	541
track	music:Track	8,664
concert	music:Concert	52
venue	music:Venue	15
class	owl:Class	57
propert	rdf:Property	40+

Dependencies and Setup

The application requires only `rdflib` (see `requirements.txt`):

```
1 rdflib
```

Listing 28: requirements.txt

Installation and execution:

```
1 pip install rdflib
2 python rag_app.py
```

Listing 29: Setup and run

Limitations

- **No LLM-based answer generation:** The application retrieves and displays information directly from the knowledge graph, including entity names, counts, classes, and properties. Although retrieval functionality is implemented, responses are not yet transformed into fully natural-language explanations using a Large Language Model (LLM) such as GPT or Claude.
- **Rule-based intent detection:** User questions are matched using keyword-based `if/elif` rules. This approach works for simple queries but may fail for complex, ambiguous, or differently phrased questions. A production-ready system would benefit from semantic parsing or machine-learning-based intent classification.
- **Limited query coverage:** The current implementation supports a predefined set of entity types and aggregation queries (artists, albums, tracks, concerts, venues, classes, and properties). More advanced competency questions would require additional SPARQL templates and query logic.
- **No multi-hop reasoning:** Questions such as “Which awards were received by artists signed to a particular record label?” require multiple graph traversals and more sophisticated SPARQL queries than those currently implemented.
- **Result size limitation:** Entity-listing queries use `LIMIT 10` to keep command-line output readable. While suitable for demonstration purposes, this restricts access to the full set of matching results.

- **Standalone RDFLib execution:** The RAG application operates on a locally loaded RDFLib graph rather than querying a live GraphDB endpoint. While this improves portability and simplifies deployment, it does not demonstrate real-time interaction with the deployed repository.

Summary and Conclusion

This project successfully demonstrates the complete semantic web technology stack applied to the popular music domain. Starting from a relational database, a formal OWL 2 ontology was created with 57 classes and 40+ properties, mapped to relational data via Ontop OBDA, materialised into 72,419 RDF triples, validated with SHACL (0 violations), aligned to an external ontology (PMO/BIBFRAME), and served through a prototype RAG application.

Table 9: Project Deliverables Summary

Artifact	File	Key Metrics
OWL 2 Ontology	music_ontology.owl	57 classes, 40+ properties
Enhanced OWL Ontology	music_ontology_enhanced.owl	Extended version
OBDA Mapping File	music_ontology.obda	11 mapping declarations
DB Connection Properties	music_ontology.properties	H2 JDBC configuration
Materialised KG	music_ontology-materialized.ttl	72,419 RDF triples, 9.09 MB
SHACL Shapes (main)	music_shapes.ttl	8 node shapes
SHACL Shapes (concert)	concert_shape.ttl	1 detailed concert shape
SHACL Validation Report	validation_report.ttl	sh:conforms true
Ontology Alignment	music_alignment.rdf	3 mappings, AML format
Performed Music Ontology	PerformedMusicOntology.owl	Reference alignment target
RAG Application	rag_app.py	Python CLI, rdflib-based

Key Design Decisions

- **Genre as a first-class entity:** Modelling **Genre** as an OWL class (not a string literal) and providing genre subclasses enables multi-genre tagging, genre-level SPARQL queries, and genre-level reasoning.
- **Dynamic venue and artist typing via OBDA:** The OBDA mappings use {venue_type} and {musician_type} template variables to dynamically assign OWL subclass types during materialisation, without requiring separate mappings for each subtype.
- **Property chain for instrument inference:** The featuresInstrument property chain (hasMember ◦ playsInstrument) allows OWL reasoners to automatically infer what instruments a band uses, without redundant data entry.
- **Typed Situation ODP:** Situations (AwardSituation, MembershipSituation) allow complex contextual facts to be expressed as first-class entities, enabling richer queries than simple binary triples.
- **In-process RDF loading in RAG:** Loading the TTL directly into RDFLib removes the dependency on a running GraphDB server during development, making the application portable and self-contained.
- **Null-safe label-to-label mapping:** The award–artist mapping uses WHERE label_id IS NOT NULL to prevent null key errors, ensuring the materialised graph remains clean.

References

- W3C. *OWL 2 Web Ontology Language Primer*, 2012.
<https://www.w3.org/TR/owl2-primer/>
- W3C. *R2RML: RDB to RDF Mapping Language*, 2012.
<https://www.w3.org/TR/r2rml/>
- W3C. *Shapes Constraint Language (SHACL)*, 2017.
<https://www.w3.org/TR/shacl/>
- Ontop SPARQL-to-SQL & OBDA Engine.
<https://ontop-vkg.org/>
- GraphDB 10.x Documentation, Ontotext.
<https://graphdb.ontotext.com/>
- AgreementMakerLight (AML).
<https://github.com/AgreementMakerLight/AML-Project>
- Performed Music Ontology (PMO).
<http://performedmusicontology.org/ontology/>
- Library of Congress BIBFRAME Ontology.
<https://id.loc.gov/ontologies/bibframe.html>
- Ontology Design Patterns Portal.
<http://ontologydesignpatterns.org>
- OOPS! Ontology Pitfall Scanner.
<http://oops.linkeddata.es/>
- TopQuadrant SHACL Engine.
<https://github.com/TopQuadrant/shacl>
- RDFLib Python Library.
<https://rdflib.readthedocs.io/>
- H2 Database Engine.
<https://h2database.com/>